

# Hardware Software Co-Design and Program Modeling

## Traditional Embedded System Development Approach

- ✓ The hardware software partitioning is done at an early stage
- ✓ Engineers from the software group take care of the software architecture development and implementation, and engineers from the hardware group are responsible for building the hardware required for the product
- ✓ There is less interaction between the two teams and the development happens either serially or in parallel and once the hardware and software are ready, the integration is performed

# Hardware Software Co-Design and Program Modeling

## Hardware Software Co-design Approach for Embedded System Development

- ✓ The product requirements captured from the customer are converted into system level needs or processing requirements rather than partitioning them to either h/w or s/w
- ✓ The system level processing requirements are then transferred into functions which can be simulated and verified against performance and functionality
- ✓ The Architecture design follows the system design. The partition of system level processing requirements into hardware and software takes place during the this phase
- ✓ Each system level processing requirement is mapped as either hardware and/or software requirement
- ✓ The partitioning is performed based on the hardware-software trade-offs

# Hardware Software Co-Design and Program Modeling

## Fundamental issues in H/w S/w Co-design

### ❑ Model Selection

- ✓ A Model captures and describes the system characteristics
- ✓ A model is a formal system consisting of objects and composition rules
- ✓ It is hard to make a decision on which model should be followed in a particular system design.
- ✓ Most often designers switch between a variety of models from the requirements specification to the implementation aspect of the system design
- ✓ The objectives vary with each phase

# Architecture Selection

- ✓ A model only captures the system characteristics and does not provide information on ‘how the system can be manufactured?’
- ✓ The architecture specifies how a system is going to implement in terms of the number and types of different components and the interconnection among them
- ✓ Controller architecture,
- ✓ Datapath Architecture,
- ✓ Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC),
- ✓ Very long Instruction Word Computing (VLIW),
- ✓ Single Instruction Multiple Data (SIMD), Multiple Instruction Multiple Data (MIMD) etc are the commonly used architectures in system design

# Language Selection

- ✓ A programming Language captures a ‘Computational Model’ and maps it into architecture
- ✓ A model can be captured using multiple programming languages like C, C++, C#, Java etc for software implementations and languages like VHDL, System C, Verilog etc for hardware implementations
- ✓ Certain languages are good in capturing certain computational model. For example, C++ is a good candidate for capturing an object oriented model.
- ✓ The only pre-requisite in selecting a programming language for capturing a model is that the language should capture the model easily

## Partitioning of System Requirements into H/w and S/w

### Implementation aspect of a System level Requirement

- ✓ It may be possible to implement the system requirements in either hardware or software (firmware)
- ✓ Various hardware software trade-offs like performance, re-usability, effort etc are used for making a decision on the hardware-software partitioning

# Hardware Software Co-Design and Program Modeling

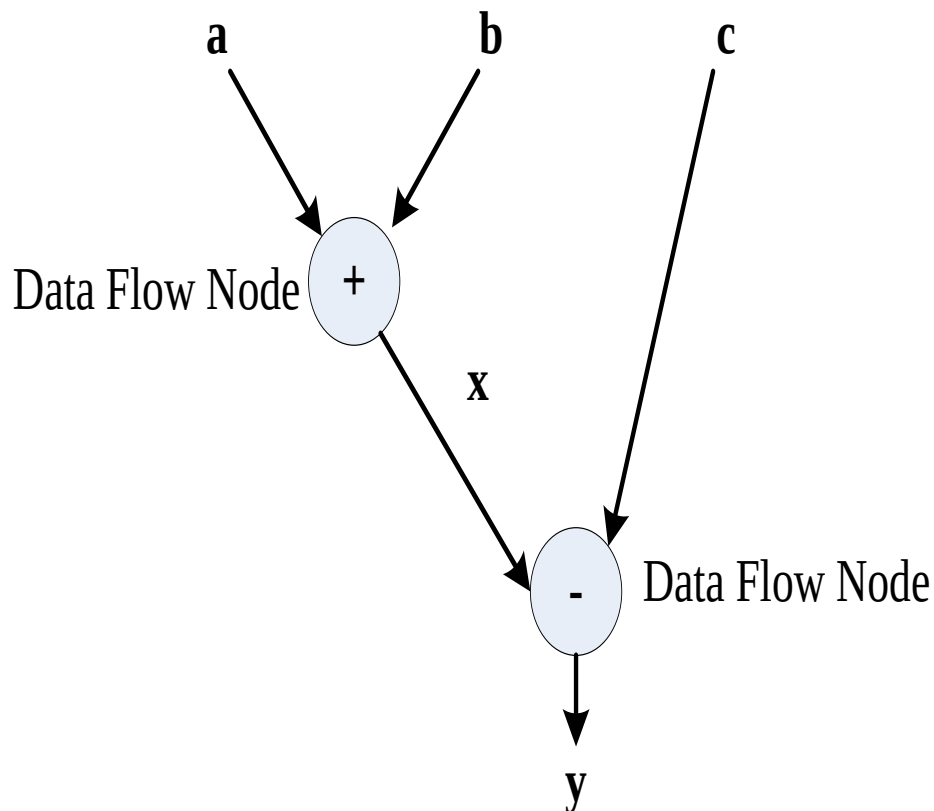
## Computational Models in Embedded Design

### Data Flow Graph/Diagram (DFG) Model

- ✓ Translates the data processing requirements into a data flow graph
- ✓ A data driven model in which the program execution is determined by data.
- ✓ Emphasizes on the data and operations on the data which transforms the input data to output data.
- ✓ A visual model in which the operation on the data (process) is represented using a block (circle) and data flow is represented using arrows. An inward arrow to the process (circle) represents input data and an outward arrow from the process (circle) represents output data in DFG notation
- ✓ Best suited for modeling Embedded systems which are computational intensive (like DSP applications)

## Computational Models in Embedded Design – Data Flow Graph/Diagram (DFG) Model

E.g. Model the requirement  $x = a + b$ ; and  $y = x - c$ ;



**Data path:** The data flow path from input to output

A DFG model is said to be acyclic DFG (ADFG) if it doesn't contain multiple values for the input variable and multiple output values for a given set of input(s). Feedback inputs (is feed back to Input), events etc are examples for non-acyclic inputs. A DFG model translates the program as a single sequential process execution.

# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design

### ❑ Control Data Flow Graph/Diagram (CDFG) Model

- ✓ Translates the data processing requirements into a data flow graph
- ✓ Model applications involving conditional program execution
- ✓ Contains both data operations and control operations
- ✓ Uses Data Flow Graph (DFG) as element and conditional (constructs) as decision makers.
- ✓ CDFG contains both data flow nodes and decision nodes, whereas DFG contains only data flow nodes

# Hardware Software Co-Design and Program Modeling

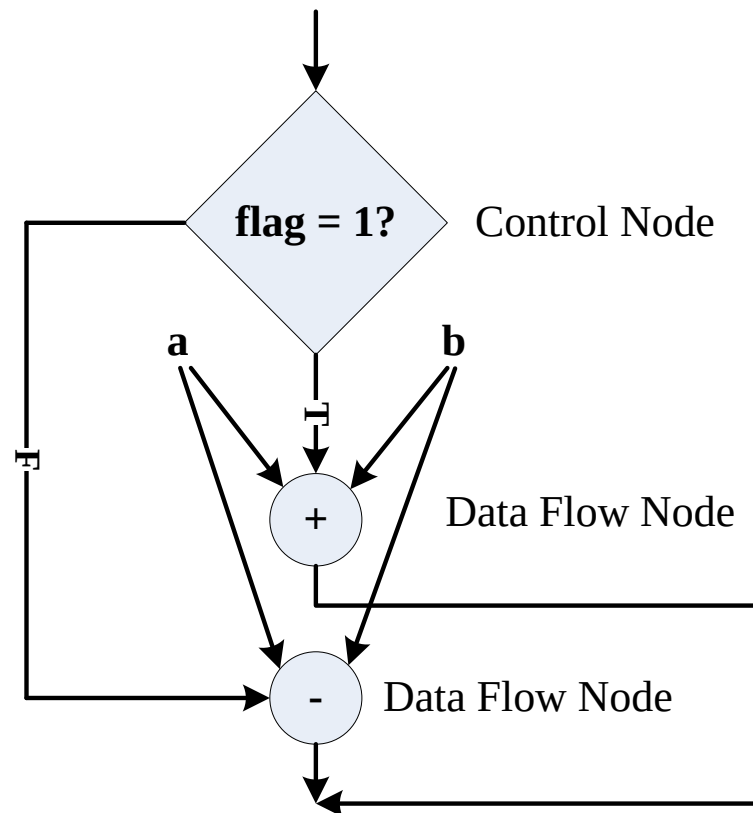
- ✓ A visual model in which the operation on the data (process) is represented using a block (circle) and data flow is represented using arrows. An inward arrow to the process (circle) represents input data and an outward arrow from the process (circle) represents output data in DFG notation.
- ✓ The control node is represented by a 'Diamond' block which is the decision making element in a normal flow chart based design
- ✓ Translates the requirement, which is modeled to a concurrent process model
- ✓ The decision on which process is to be executed is determined by the control node

# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design –

### Control Data Flow Graph/Diagram (CDFG) Model

E.g. Model the requirement If flag = 1,  $x = a + b$ ; else  $y = a - b$ ;



# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design

### ❑ State Machine Model

- ✓ Based on ‘States’ and ‘State Transition’
- ✓ Describes the system behavior with ‘States’, ‘Events’, ‘Actions’ and ‘Transitions’
- ✓ *State* is a representation of a current situation.
- ✓ An *event* is an input to the *state*. The *event* acts as stimuli for state transition.
- ✓ *Transition* is the movement from one state to another.
- ✓ *Action* is an activity to be performed by the state machine.

# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design

### ❑ State Machine Model

- ✓ A Finite State Machine (FSM) Model is one in which the number of states are finite. In other words the system is described using a finite number of possible states
- ✓ Most of the time State Machine model translates the requirements into sequence driven program
- ✓ The Hierarchical/Concurrent Finite State Machine Model (HCFSM) is an extension of the FSM for supporting concurrency and hierarchy

# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design

### ❑ State Machine Model

- ✓ HCFSM extends the conventional state diagrams by the AND, OR decomposition of States together with inter level transitions and a broadcast mechanism for communicating between concurrent processes
- ✓ HCFSM uses statecharts for capturing the states, transitions, events and actions. The Harel Statechart, UML State diagram etc are examples for popular statecharts used for the HCFSM modeling of embedded systems

# Hardware Software Co-Design and Program Modeling

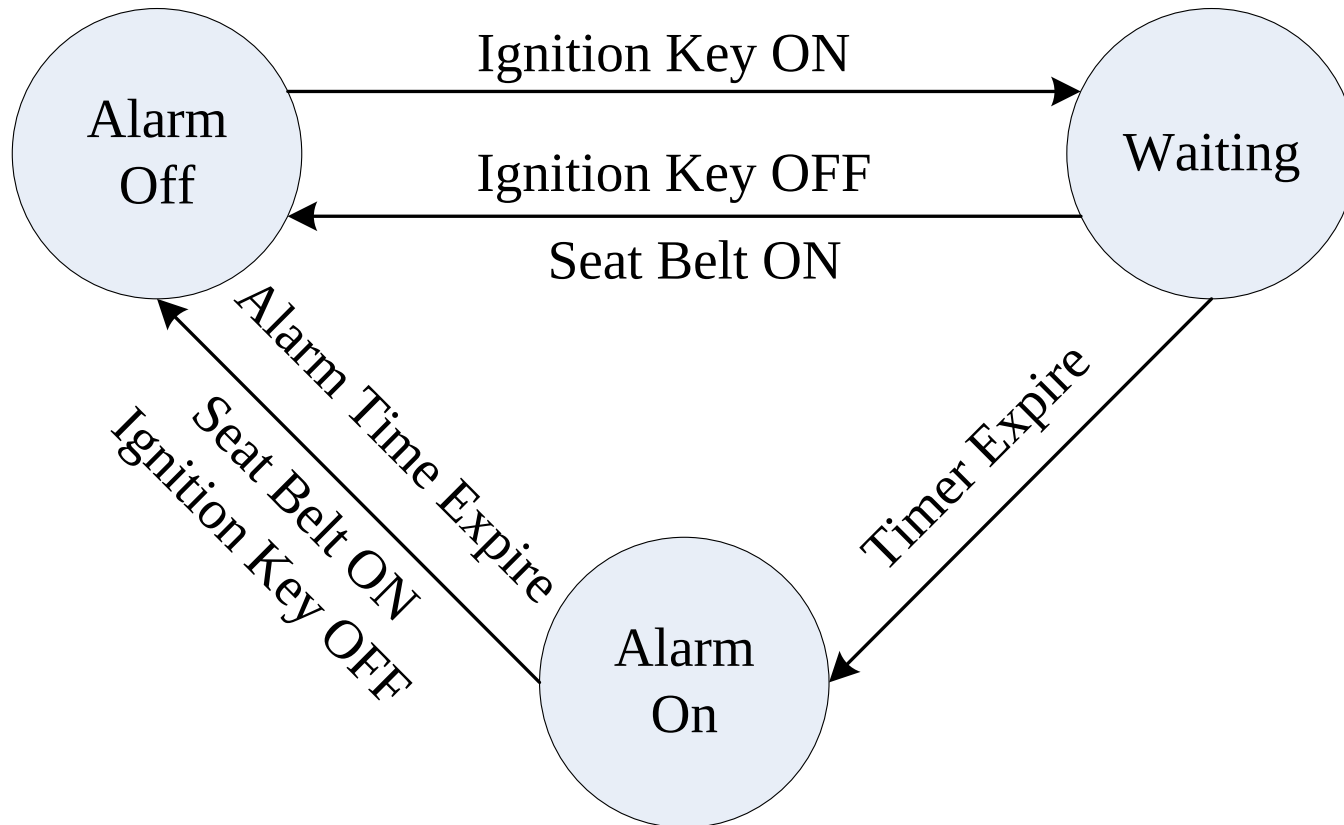
## Computational Models in Embedded Design – Finite State Machine (FSM) Model

E.g. Automatic ‘Seat Belt Warning’ in an automotive

### **Requirement:**

- When the vehicle ignition is turned on and the seat belt is not fastened within 10 seconds of ignition ON, the system generates an alarm signal for 5 seconds.
- The Alarm is turned off when the alarm time (5 seconds) expires or if the driver/passenger fastens the belt or if the ignition switch is turned off, whichever happens first.

# Hardware Software Co-Design and Program Modeling



# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design

### ❑ Sequential Program Model

- ✓ The functions or processing requirements are executed in sequence
- ✓ The program instructions are iterated and executed conditionally and the data gets transformed through a series of operations
- ✓ FSMs are good choice for sequential Program modeling.
- ✓ Flow Charts is another important tool used for modeling sequential program
- ✓ The FSM approach represents the states, events, transitions and actions, whereas the Flow Chart models the execution flow

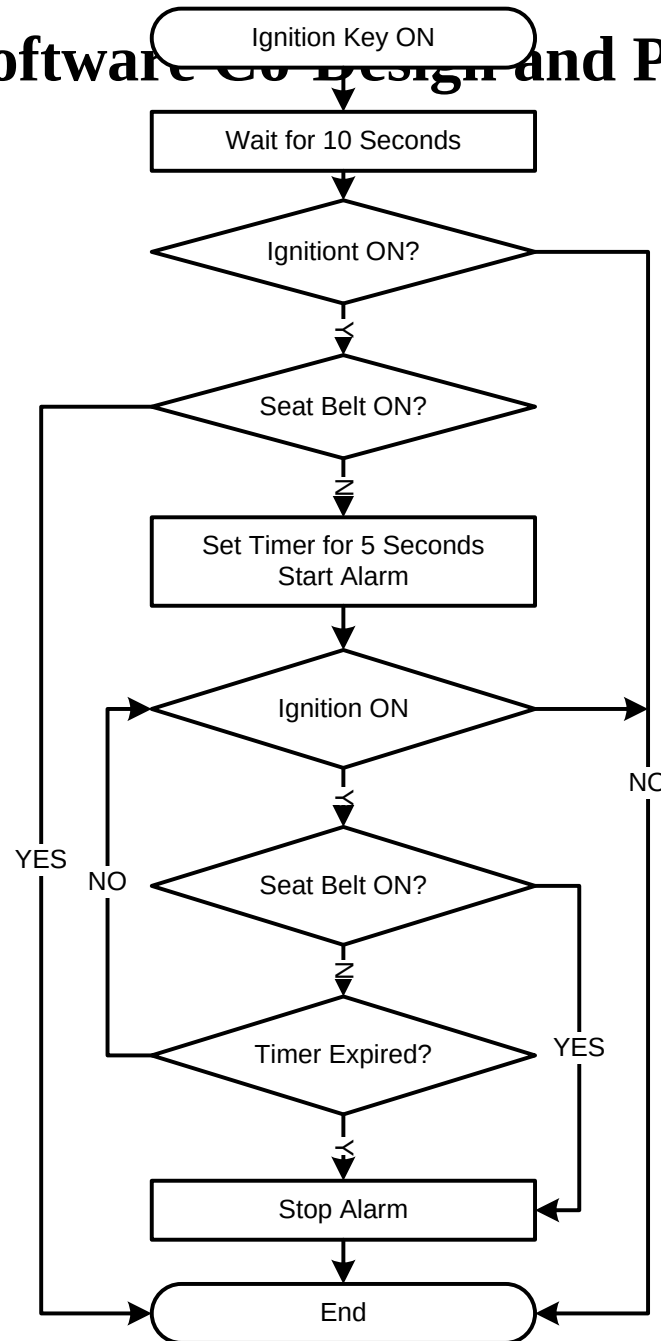
# Computational Models in Embedded Design –Sequential Program Model

E.g. Automatic ‘Seat Belt Warning’ in an automotive

## **Requirement:**

- When the vehicle ignition is turned on and the seat belt is not fastened within 10 seconds of ignition ON, the system generates an alarm signal for 5 seconds.
- The Alarm is turned off when the alarm time (5 seconds) expires or if the driver/passenger fastens the belt or if the ignition switch is turned off, whichever happens first.

# Hardware Software Co-Design and Program Modeling



# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design –Sequential Program Model

E.g. Automatic ‘Seat Belt Warning’ in an automotive

```
#define ON 1
#define OFF 0
#define YES 1
#define NO 0
void seat_belt_warn()
{
    wait_10sec();
    if (check_ignition_key()==ON)
    {
        if (check_seat_belt()==OFF)
        {
            set_timer(5);
            start_alarm();
            while ((check_seat_belt()==OFF )&&(check_ignition_key()==OFF )&&
                (timer_expire()==NO));
            stop_alarm();
        }
    }
}
```

# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design

### ❑ Concurrent/Communicating Process Model

- ✓ Models concurrently executing tasks/processes The program instructions are iterated and executed conditionally, and the data gets transformed through a series of operations
- ✓ Certain processing requirements are easier to model in concurrent processing model than the conventional sequential execution.
- ✓ Sequential execution leads to a single sequential execution of task and thereby leads to poor processor utilization, when the task involves I/O waiting, sleeping for specified duration etc.
- ✓ If the task is split into multiple subtasks, it is possible to tackle the CPU usage effectively, when the subtask under execution goes to a wait or sleep mode, by switching the task execution.
- ✓ Concurrent processing model requires additional overheads in task scheduling, task synchronization and communication

# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design – Concurrent Processing Model

E.g. Automatic ‘Seat Belt Warning’ in an automotive

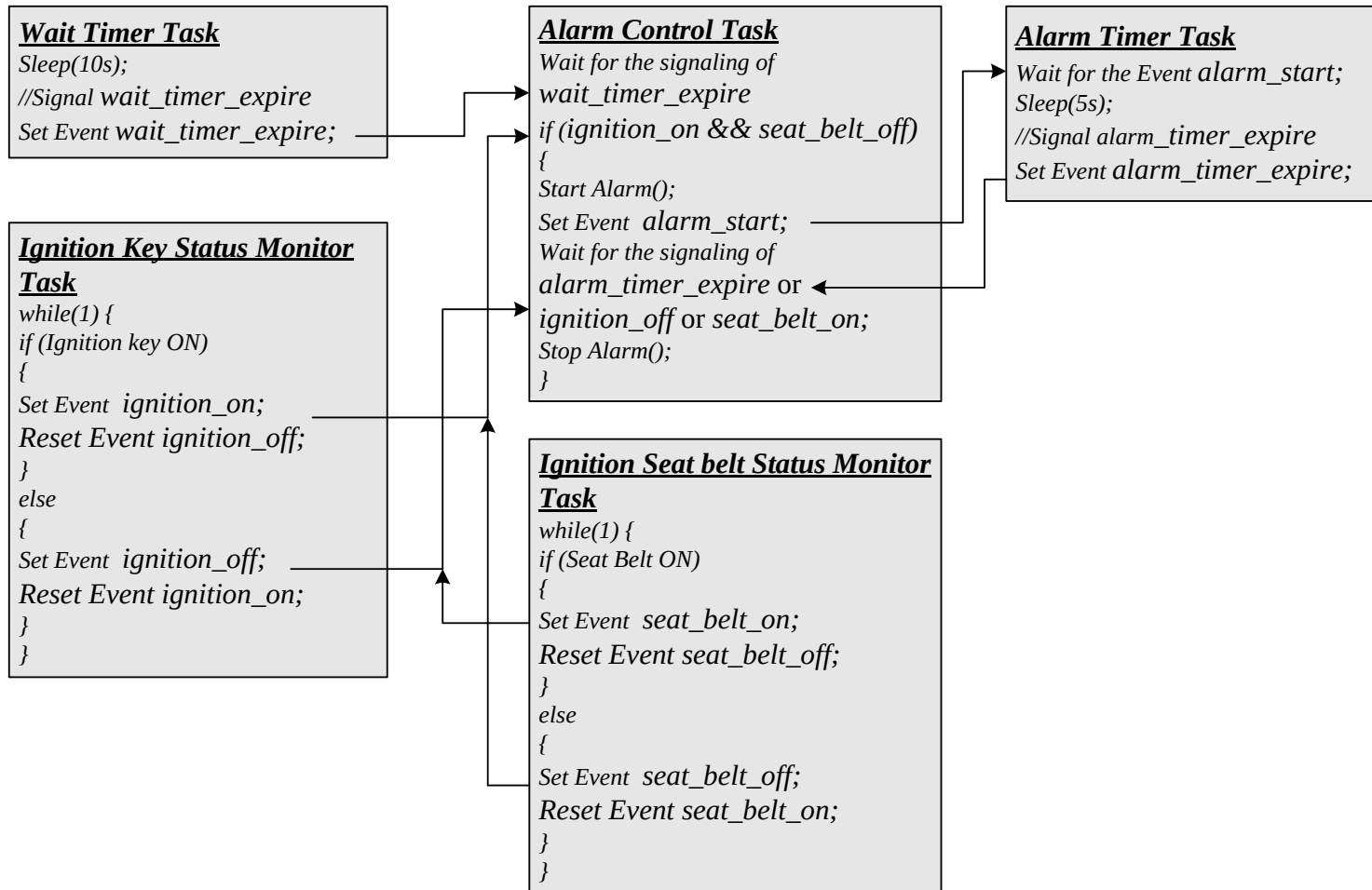
Create and initialize events  
*wait\_timer\_expire, ignition\_on, ignition\_off,*  
*seat\_belt\_on, seat\_belt\_off,*  
*alarm\_timer\_start, alarm\_timer\_expire*  
Create task *Wait Timer*  
Create task *Ignition Key Status Monitor*  
Create task *Seat Belt Status Monitor*  
Create task *Alarm Control*  
Create task *Alarm Timer*

(a)

# Hardware Software Co-Design and Program Modeling

## Computational Models in Embedded Design – Concurrent Processing Model

E.g. Automatic ‘Seat Belt Warning’ in an automotive

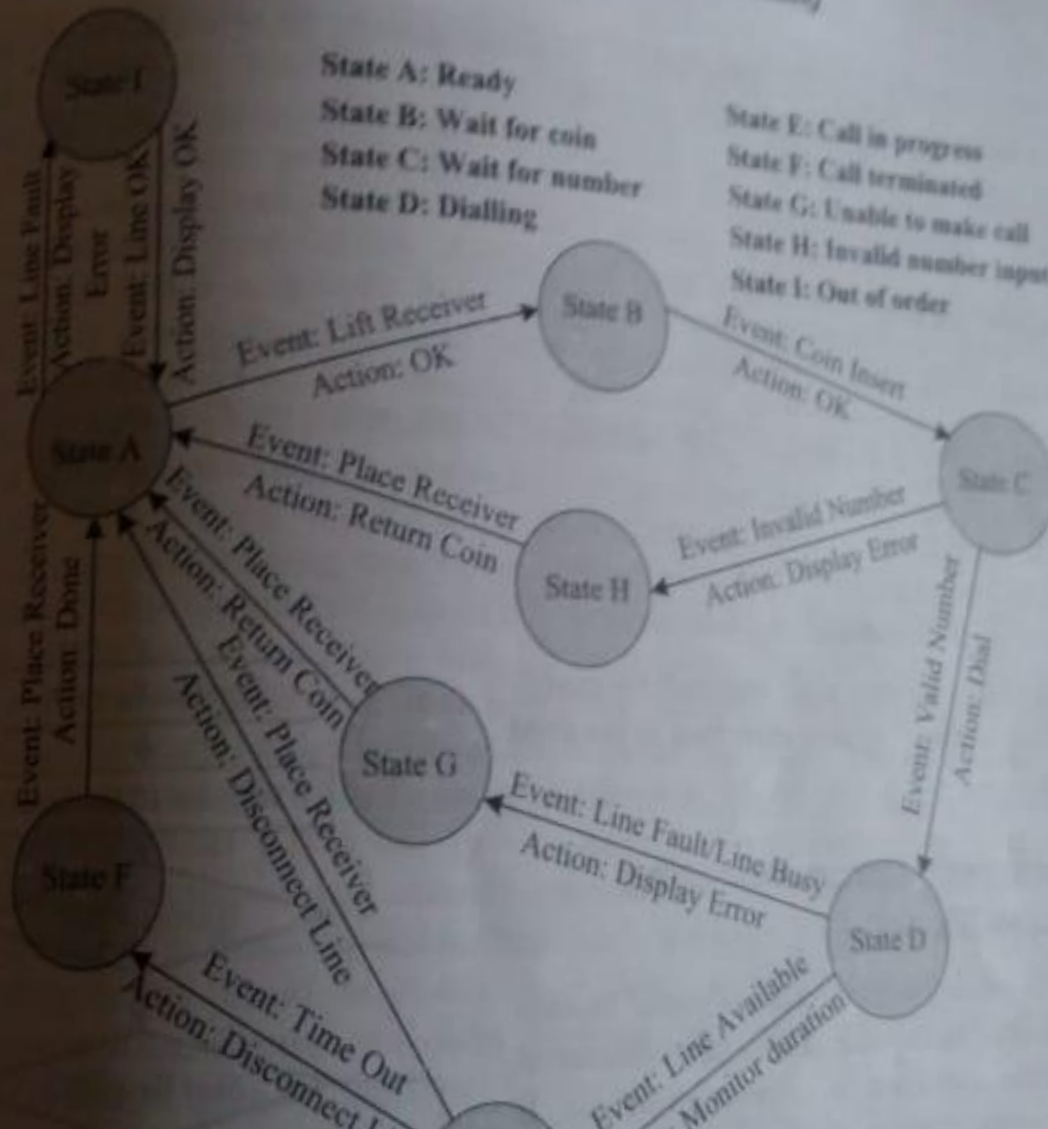


(b)

- ✓ The processing requirements are split in to multiple tasks
- ✓ Tasks are executed concurrently
- ✓ ‘Events’ are used for synchronizing the execution of tasks

# Hardware Software Co-Design and Program Modelling

211



Wait for coin  
 Wait for user input  
 Dispense tea  
 Dispense coffee

for the 'Wait State' (Current input is received within for Coin' on the timeout and changing the state to